

A Project Synopsis Presentation on

“AI-Driven Automated Web Application Penetration Testing Tool “ (AWAP-AI)

Presented By:

Deepak J M U23E01CY007

Krupakar S Billichod U23E01CY013

Ranjith kumar S U23E01CY023

Uder Guidance of:

Ms. Rakshitha G B

Associate Professor,
Dept. of CY & IY

Project Coordinator:

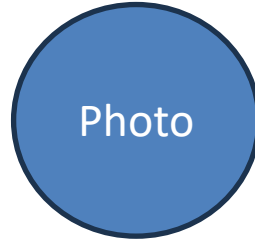
Dr. Rachana P G

Assistant Professor,
Dept. of CY & IY

Program Coordinator.

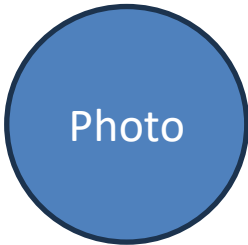
Dr. Aruna Kumar B T

Associate Professor &HOD,
Dept. of CY & IY



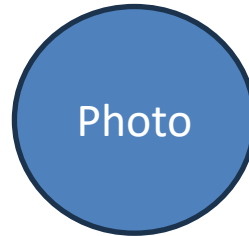
Photo

Ms. Rakshitha G B



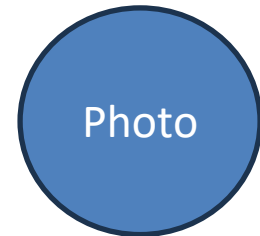
Photo

Deepak J M



Photo

Krupakar S Billichod



Photo

Ranjith kumar S

Contents

- Introduction
- Literature Survey
- Problem Statement
- Objectives
- Requirements
- Methodology
- Dataset /Coding
- Results/Expected Outcomes
- Conclusion
- Future work
- Reference

Introduction

- Web applications represent the primary attack surface of the modern internet.
- Traditional vulnerability scanners rely on static rule-matching, which are inadequate against modern Single-Page Applications (SPAs) and dynamic APIs.
- AWAP-AI is a full-stack autonomous web application penetration testing tool.
- It combines classical penetration testing methodology with an AI layer that reasons about attack paths, generates context-aware payloads, and evaluates vulnerabilities.

Problem Statement

1. Problem Definition:

- Modern web applications are massive and highly complex to secure.
- Attackers adapt rapidly; defenders use slow, fixed-rule tools.
- Single point-in-time scans create a backlog of undetected vulnerabilities.

2. Expected Outcome:

- An autonomous AI-driven tool capable of executing all phases of a security assessment such as reconnaissance, enumeration, vulnerability identification, and reporting.
- Zero manual intervention required between testing phases.

Literature Survey

Sl No	Title of the Paper	Author Name	Year of Published	Technology Adopted
1	Securing Web Applications from Injection and Logic Vulnerabilities	K. Smith et al.	2017	Dynamic Application Security Testing
2	Machine learning supervised algorithms for anomaly detection	Abdulqader et al.	2020	Supervised ML Logic
3	Crowdsourced vs. Automated Penetration Testing	Zhao et al.	2022	Automation Statistical Analysis
3	GPT-4 for Automated Vulnerability Detection	Pearce et al.	2023	Generative AI & NLP
3	LLM Agents can Autonomously Hack Websites	Richard et al.	2024	Large Language Models
4	Hacker-Powered Security Report	HackerOne	2024	Bug Bounty AI Automation

Objectives

1. **To Design** an automated tool architecture utilizing multiple specialized engines (Recon, Crawler, Attack) orchestrated via a central message bus.
2. **To Develop** intelligent payload generation engines and Machine Learning classifiers that accurately analyze HTTP responses and detect anomalies.
3. **To Implement** a comprehensive testing tool that produces professional, CVSS-scored findings with minimal false positives.

Requirements

1. Hardware Requirements:

- ✓ **CPU:** Dual-core Processor (Intel i5 or equivalent)
- ✓ **RAM:** 8 GB minimum
- ✓ **Storage:** 256 GB SSD

2. Software Requirements:

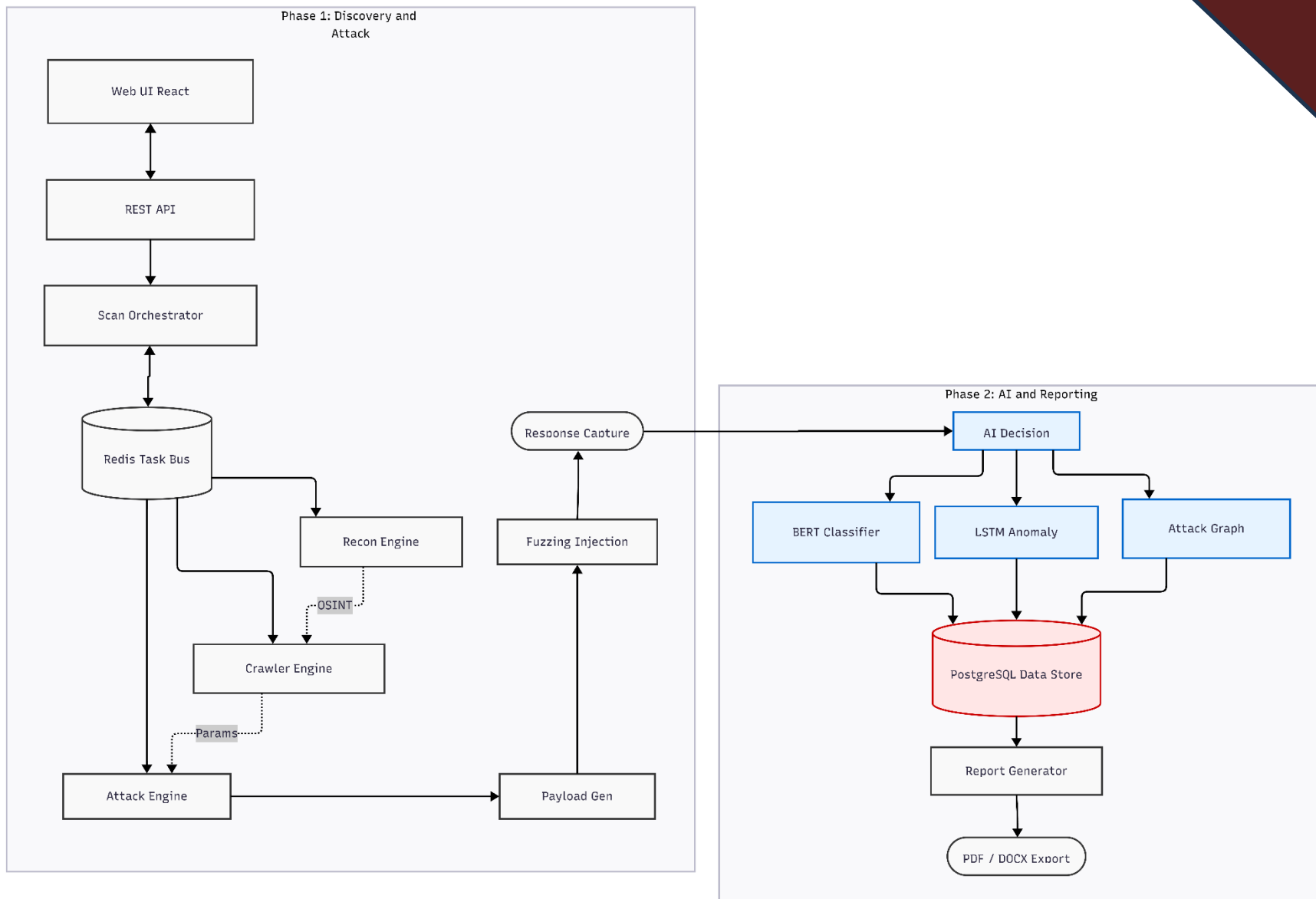
- ✓ **Backend:** Python 3.11+, FastAPI, Celery, Playwright
- ✓ **Core Databases:** PostgreSQL 16 (Data Store), Redis 7 (Message Bus/Queues)
- ✓ **Frontend:** React 18 with TypeScript, Tailwind CSS
- ✓ **AI/ML Stack:** HuggingFace Transformers (BERT), PyTorch, scikit-learn
- ✓ **Infrastructure:** Docker Compose for local development

Methodology

The tool operates via a seamless workflow pipeline:

1. **Scope Verification:** Normalizes the target and logs authorization.
2. **Reconnaissance:** DNS enumeration, port scanning, and OSINT gathering.
3. **Crawl & Mapping:** Headless Playwright crawling extracting APIs and parameters.
4. **Attack Execution:** Running vulnerability modules (SQLi, XSS, CMDi, SSRF) utilizing a 5-layer mutation engine.
5. **AI Analysis:** Using BERT and Isolation Forests to classify findings and reduce false positives.
6. **Reporting:** Generating CVSS-scored outputs.

Flowchart



Dataset /Coding

AI / ML Dataset:

- Uses HuggingFace deep learning models.
- Fine-tuned on massive CVE and HackerOne vulnerability datasets.
- Implements Isolation Forests for anomaly detection.
- Uses Thompson Sampling for adaptive scan prioritization.

Coding Stack:

- **Backend:** Python (FastAPI), Celery distributed task queues.
- **Messaging:** Redis Streams.
- **Frontend:** React.js dashboard with Zustand state management.

Results

- A professional, SOC-aesthetic live dashboard.
- Real-time scan progress streaming using native WebSockets.
- Instant classification of High/Critical vulnerabilities via AI inference.
- Auto-generated Proof of Concept (PoC) code for developers.
- Downloadable security reports in PDF/JSON/DOCX formats.

Conclusion

- Security testing must evolve beyond standard signature scans.
- This tool successfully simulates a real bug bounty researcher's workflow.
- Orchestrates advanced web crawling and intelligent fuzzing seamlessly.
- Deep Learning response classification discovers flaws efficiently and reliably.

Future work

- Expand LLM capabilities to auto-write custom exploit scripts.
- Integrate directly into Enterprise CI/CD pipelines (e.g., GitHub Actions).
- Cloud-native deployment via Kubernetes for massive horizontal scaling.

Reference

- OWASP Top 10 – Web Application Vulnerabilities: <https://owasp.org/Top10/>
- OWASP Web Security Testing Guide: <https://owasp.org/www-project-web-security-testing-guide/>
- CWE – Common Weakness Enumeration: <https://cwe.mitre.org/>
- CVE – Common Vulnerabilities and Exposures: <https://www.cve.org/>
- Cybersecurity Research Papers: <https://arxiv.org/>
- *Securing Web Applications from Injection and Logic Vulnerabilities: Approaches and Challenges - Information and Software Technology*, 96, 210-230: <https://doi.org/10.1016/j.infsof.2017.11.010>
- *Hacker-Powered Security Report 2024* – HackerOne:
<https://www.hackerone.com/resources/hackerone/8th-annual-hacker-powered-security-report>
- *LLM Agents can Autonomously Hack Websites* — arXiv preprint arXiv:2402.06664
<https://arxiv.org/abs/2402.06664>

THANK YOU